Offline-First Smart Home Node with Hybrid Local–Cloud Autonomy and Adaptive Energy Management

Le Xuan Quyet (SE203751), Nguyen Quoc Viet (SE211888), Nguyen Hoang Quan (SE204702), and Tran Quoc Huy (SE196671)

Abstract

Most smart-home IoT prototypes depend on continuous cloud connectivity and transmit sensor data at fixed intervals, so scheduled actions fail during network outages and radio energy is wasted when conditions are static. This report presents the design of an offline-first smart-home node built around a single ESP8266. The node combines a hardware real-time clock (RTC) for local scheduling, event-driven (threshold-based) environmental reporting, rule-based fan and light control, and two-factor offline access control (RFID card and PIN), and it synchronizes schedules and logs with the cloud whenever a connection is available. The report reviews seven related studies, identifies three research gaps, states one main and three sub research questions, and describes the system architecture, the hardware interfacing plan, the firmware design and a quantitative evaluation plan covering scheduling deviation, reconciliation after an outage, message reduction, energy consumption and authentication response time. The prototype uses low-cost components from the IOT102 course kit and a few additional modules. At the first milestone the node is built on a breadboard and the sensors, the local control rules, the card-based access control and the event logging to the cloud are operational; the PIN factor, the offline queue and the quantitative measurements follow at the next milestones.

Index Terms

Internet of Things, smart home, ESP8266, offline-first, RTC scheduling, adaptive sampling, MQTT.

I. Introduction

The Internet of Things (IoT) makes it possible to monitor and control everyday household devices remotely. A smart home typically senses the state of a room (temperature, humidity, light, gas and motion), processes the data on a microcontroller, and exposes the result through a messaging protocol and a dashboard. Four functional domains recur in practice: time-based and remote control, environmental monitoring and energy efficiency, security and access control, and cloud integration.

Existing prototypes usually address one or two of these domains and share three weaknesses. First, scheduling is often a cloud feature or a simple countdown timer, so a network outage stops automation. Second, sensors are sampled and uploaded at a fixed rate, which wastes radio energy when conditions do not change. Third, authentication frequently needs the cloud, so users can be locked out when the network fails. These weaknesses matter most in a home, where a lamp, a fan or a door lock is expected to keep working when the internet does not.

This report designs and will evaluate a single-node smart-home system that keeps its essential functions running locally. Its contributions are:

1) an offline-first architecture in which a hardware RTC executes schedules locally and the node reconciles its state with the cloud after an outage;
2) event-driven reporting at the edge, which sends sensor data only on significant change or at a heartbeat interval;
3) two-factor access control (RFID card and PIN) whose credentials are verified from local storage;
4) a reproducible low-cost prototype built around a single ESP8266 from the IOT102 course kit and a few additional modules, together with a quantitative evaluation plan.

The prototype is a one-room (or model-room) setup and switches low-voltage loads only; mains-powered appliances are out of scope for safety reasons.

II. Related Work

A. Time-Based and Remote Control

Haroon et al. [1] control household appliances with an ESP32, relays and a DHT11 sensor through the Blynk platform and report an average response latency below 500 ms. Their timer function switches a relay off after a user-defined duration, which is a countdown rather than a recurring schedule driven by a clock. The authors also list compatibility with existing home infrastructure, power optimization and scalability as open issues. Makatita and Hakim [2] combine an ESP32, MQTT, Node-RED and an Android MQTT dashboard with DHT22, LDR and sound sensors, offering Auto and Manual modes with an average latency of 0.67–0.82 s. Their system has no time-based scheduling, and the control flow described does not go beyond repeated reconnection attempts when the connection is lost.

Group 1, Class IA2103, IOT102 – Fall 2026, FPT University, Ho Chi Minh City, Vietnam. Instructor: Tran Cong Doi.

TABLE I
SUMMARY OF RELATED STUDIES

Study	Technology	Limitation relevant to this work
Haroon et al. (2024) [1]	ESP32, relay, DHT11, Blynk	Countdown timer only; no recurring RTC schedule; third-party cloud
Makatita and Hakim (2024) [2]	ESP32, MQTT, Node-RED, DHT22, LDR, KY-038	No time scheduling; no described behavior during an outage
Sulimat and Katiran (2024) [3]	Arduino Uno, LoRaWAN, ThingSpeak, solar Li-ion	Fixed 30 s update; no sleep or adaptive sampling; prototype
Parkavi et al. (2025) [4]	NodeMCU, MQ135, dust sensor, DHT11, ARIMA	Fixed 120 s upload; no change-based reporting
Alzhrani et al. (2024) [5]	ESP32, YOLOv3, fingerprint, PIR, Telegram	Cloud takes part in credential checks
Parab and Kamat (2022) [6]	NodeMCU, optical fingerprint, solenoid	Waits for Wi-Fi at start-up; single factor; preprint
Alfazri et al. (2025) [7]	PIR, SW-420, servo/solenoid, IP camera, Firebase, Telegram	Depends on stable internet; no environmental sensing reported; offline storage listed as future work

B. Environmental Monitoring and Energy Efficiency

Sulimat and Katiran [3] present a portable air-quality monitor based on an Arduino Uno with a LoRa shield, a DHT11, MQ9, MQ135 and a dust sensor. Data pass through The Things Network to ThingSpeak, and the data fields refresh every 30 s. The device is powered by a solar-charged Li-ion battery, but the paper describes no sleep mode or adaptive sampling, and the authors describe the system as a prototype. Parkavi et al. [4] monitor hospital air quality with a NodeMCU, MQ135, a Sharp dust sensor and a DHT11, upload to the cloud every 120 s, raise threshold alerts and forecast temperature and humidity with ARIMA/SARIMA models. Both systems report at a fixed period and neither describes a mechanism that sends data only when readings change.

C. Security and Access Control

Alzhrani et al. [5] build a smart locker that combines facial recognition (YOLOv3), a fingerprint sensor, emergency codes and a PIR sensor that triggers Telegram alerts, with data uploaded through a cellular link. Because the cloud takes part in checking some credentials, authentication depends on connectivity. Parab and Kamat [6] use a NodeMCU ESP8266, an optical fingerprint sensor and a solenoid lock; their start-up routine waits for Wi-Fi and does not continue until the connection is established, and the system relies on a single authentication factor.

D. Cloud Integration

Alfazri et al. [7] build a smart package-receiving box that combines a PIR motion sensor, an SW-420 vibration sensor, servo and solenoid actuators and an IP camera, and uses a REST API, a Telegram bot and a Firebase Realtime Database for monitoring, control and event logging. They report average sensor response times of 0.71–0.98 s and 100% communication reliability over Wi-Fi, the REST API and Telegram. They also note that dependence on stable internet connectivity limits the autonomy of the system, and they list local data storage for offline operation as future work. This confirms that a cloud database is widely used for state sharing and logging, and that local operation during an outage remains an open need.

E. Summary and Research Gaps

Table I summarizes the surveyed studies. In the studies reviewed, we identify three gaps.

- **G1 – Autonomous local scheduling.** Remote-control systems [1], [2] rely on a countdown timer or cloud commands, and the cloud-based system [7] depends on stable internet connectivity. We did not find a surveyed system that runs recurring schedules from a hardware RTC during an outage and then reconciles with the cloud.
- **G2 – Fixed-rate reporting.** Sensor data are sent every 30 s [3] or 120 s [4] regardless of change. Threshold-based reporting at the edge is not described in these studies.
- **G3 – Offline authentication.** Access systems [5], [6] depend on the cloud or on Wi-Fi at start-up for part of their operation, and they do not describe a locally verified fallback.

III. RESEARCH QUESTIONS AND NOVELTY

Main RQ. How can a unified smart-home IoT node integrate local RTC scheduling, event-driven environmental monitoring and two-factor local access control with cloud synchronization, while keeping deterministic operation during network disruptions?

- **RQ1 (G1).** How should a local hardware RTC and the cloud database be synchronized so that scheduled relay actions execute on time during a network failure and reconcile without state conflicts after reconnection?
- **RQ2 (G2).** By what percentage does threshold-based reporting reduce messages and energy consumption compared with fixed-interval reporting, and does it still detect significant events without noticeable delay?
- **RQ3 (G3).** Does local verification of an RFID card and a PIN against credentials stored in flash give acceptable response time and success rate on a single microcontroller during an outage?

Novelty. The contribution lies in the offline-first architecture rather than in combining components. **N1:** the RTC runs schedules locally and the node switches among online, offline and reconciliation modes (RQ1). **N2:** sensor data are sent only on significant change or at a heartbeat interval (RQ2). **N3:** scheduling, safety monitoring and two-factor authentication are integrated on one low-cost node and verified locally (RQ3).

IV. System Design

A. Requirements

The functional requirements are: (FR1) switch a fan and a light on a recurring daily schedule driven by the RTC; (FR2) allow manual override from the dashboard, with a per-device Auto/Manual mode in which manual commands take priority over schedules; (FR3) measure temperature, humidity, light level, gas level and motion; (FR4) apply occupancy rules, for example turn the light on only when the room is occupied and dark; (FR5) when the gas level exceeds a threshold, sound a buzzer, switch the fan on for ventilation and raise a dashboard alert; (FR6) grant door access only after a valid RFID card and PIN, with time-stamped logs; (FR7) show state and history on a dashboard and synchronize with the cloud; and (FR8) switch the fan on automatically when the temperature exceeds an adjustable threshold, with a hysteresis band. The non-functional requirements are continued operation of FR1, FR5, FR6 and FR8 during an outage, a single ESP8266 node, low-voltage loads only, and the use of low-cost components from the course kit.

B. Architecture Overview

Fig. 1 shows the block diagram. The sensors (DHT11, LDR, MQ2, PIR) and the access-control input (RFID RC522, joined at OPA2 by a keypad read through a second PCF8574 expander) feed the ESP8266 (NodeMCU). The INA219 current sensor of the final stage is added in series with the node supply. Power comes from a 12 V adapter through an XY-3606 converter, which provides the 5 V rail for the relay, the fan, the MQ2 heater and the servo; the ESP8266 itself is powered from USB and every ground is common. The ESP8266 drives the LCD, the relay and fan, the servo that acts as the door lock, the buzzer and the LEDs, and exchanges time data with the DS1307 RTC, which provides the time base. A PCF8574 expander extends the digital outputs and the PIR input, and a PCF8591 converter reads the LDR, because the ESP8266 has only one analog input. The node has a single network connection: an MQTT client connects to a broker (a local Mosquitto broker, or a cloud broker such as HiveMQ Cloud) and carries live commands, sensor telemetry, schedule entries, queued events and the online/offline status. Node-RED subscribes to the broker to provide the web dashboard, automation rules and logging, and it is the only component that talks to the cloud database: it mirrors schedules and event logs to a Firebase Realtime Database over HTTPS, and this copy serves as the reference during reconciliation. The ESP8266 and the devices attached to it keep working when Wi-Fi is lost; only dashboard control and cloud synchronization pause. Keeping Firebase access out of the ESP8266 keeps the firmware simple and avoids a TLS client on a device with about 80 KB of RAM. The trade-off is that Node-RED becomes the single path to Firebase: if it is down while the broker is reachable, synchronization is delayed, but the local functions of the node are not affected.

Principle of operation. Data flow from the inputs to the ESP8266, which processes them locally and drives the outputs. The ESP8266 reads the environmental sensors and sends a value to the broker only on a significant change or at a heartbeat interval. Recurring schedules for the fan and the light are executed from the RTC, and manual commands from the dashboard take priority over schedules. A door unlock requires a valid RFID card followed by a correct PIN, both verified from local storage. A gas reading above the threshold sounds the buzzer immediately, switches the fan on and raises a dashboard alert. These functions do not depend on the network. When the link is available, the node publishes telemetry and events to the MQTT broker, Node-RED shows them on the dashboard and stores schedules and logs in Firebase; when the link returns after an outage, queued events are replayed and schedules are reconciled.

C. Hardware Components

Table II lists the components and their roles. Electrical values follow the manufacturers' datasheets [8]– [14] and must be re-checked against the exact modules used.

Controller: ESP8266 (NodeMCU). The ESP8266 is a single-core 32-bit microcontroller with built-in Wi-Fi and 3.3 V logic. Its GPIOs are not 5 V tolerant. The development board exposes a single 10-bit analog input (A0) and only a handful of usable digital pins; GPIO0, GPIO2 and GPIO15 are boot-strapping pins, and GPIO16 supports plain digital output only [8]. It is the only controller of the node and runs a cooperative, timer-driven firmware, the MQTT client and the local storage.

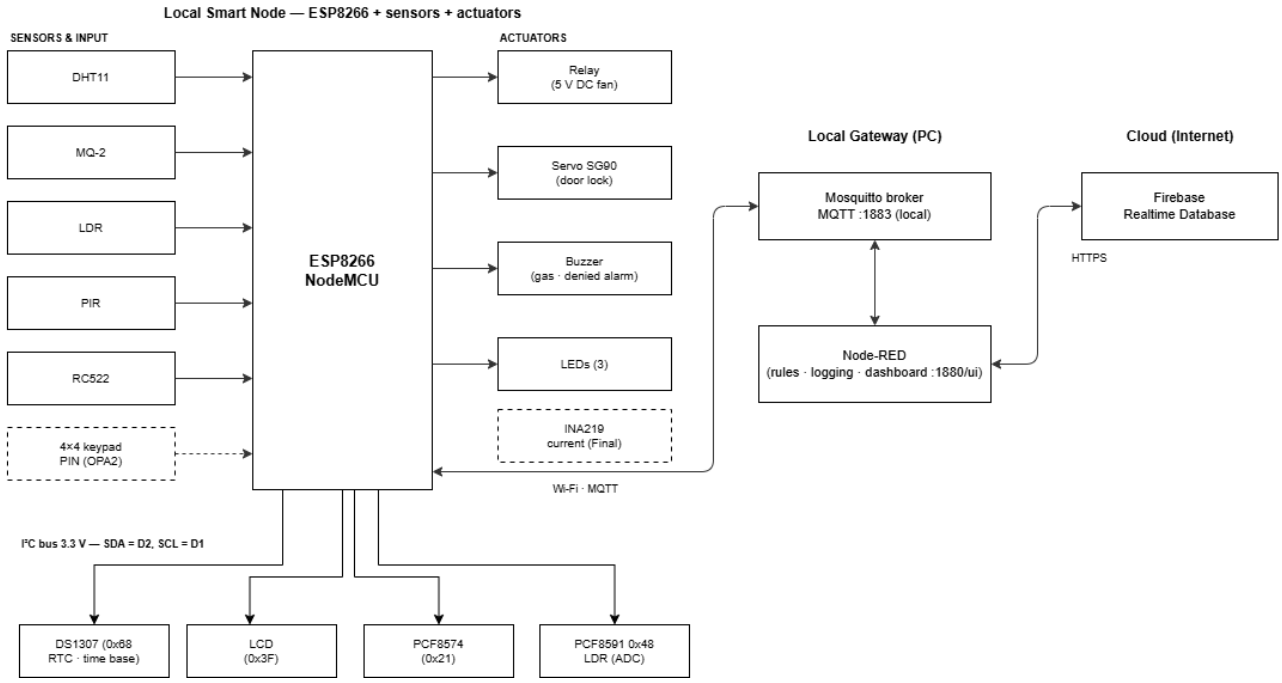


Fig. 1. Block diagram of the offline-first smart-home node. The ESP8266 and its peripherals form the local smart node; the Mosquitto broker and Node-RED run on the local gateway (the PC) and only Node-RED reaches the Internet. Dashed blocks are planned for a later stage; the 5 V rail is generated by the XY-3606 converter fed from the 12 V adapter.



(a) ESP8266 NodeMCU



(b) DS1307 RTC

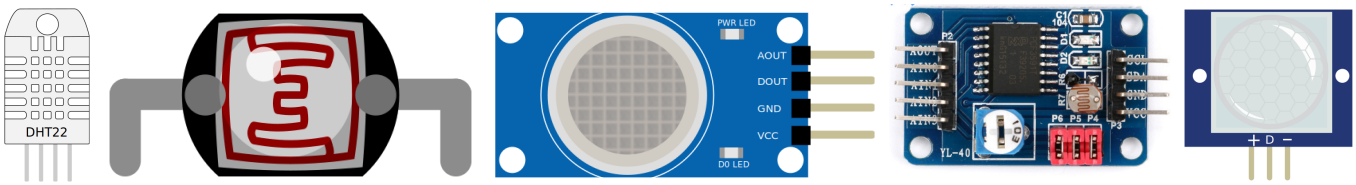
Fig. 2. Controller and time base. Replace the placeholders with images of the exact modules.

DS1307 RTC module. The DS1307 is an I2C real-time clock (address 0x68) that keeps time on a coin-cell backup input, so the time survives power and network loss [9]. It uses an external 32.768 kHz crystal without temperature compensation. Its module carries 4.7 kΩ pull-ups to its own supply rail, so it is powered from 3.3 V and shares the single 3.3 V I2C segment: with a 5 V supply those pull-ups hold SDA and SCL at 5 V and the bus fails. Its drift is larger than that of a temperature-compensated RTC such as the DS3231, which is a drop-in alternative. Instead of hiding this effect, the node records the difference between the RTC and the NTP time at every resynchronization, so that the scheduling deviation measured in RQ1 can be separated into oscillator drift and synchronization behavior. Boards with a charging circuit must be checked before a non-rechargeable coin cell is fitted.

Environmental sensors. The DHT11 is a low-cost digital sensor for temperature and relative humidity (about ±2 °C and ±5 %RH, sampling period of at least one second). The LDR (CDS 5537) forms a voltage divider with a fixed 10 kΩ resistor, and the voltage is read by a PCF8591 8-bit converter (address 0x48) [13]; an 8-bit step is sufficient for a dark/bright decision. The MQ2 is a metal-oxide sensor for smoke, LPG and other combustible gases; its heater needs 5 V and a warm-up period, its analog output is divided to at most 3.3 V and read on A0, so that the alarm path does not depend on the I2C bus, and readings are relative and are calibrated against clean air. The PIR sensor (HC-SR505 mini type) detects movement of warm bodies and provides the occupancy signal for FR4; its hold time is fixed by the module and must be checked on the actual part.

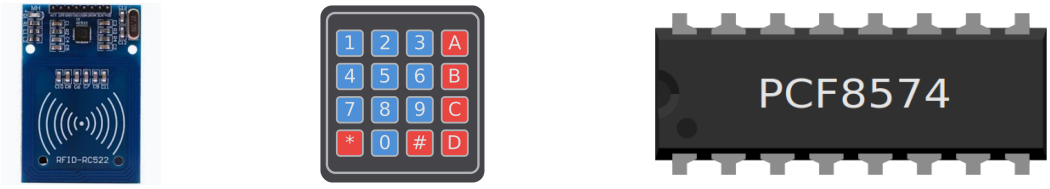
Access control. The RC522 module is based on the MFRC522 chip for 13.56 MHz MIFARE cards and is connected through SPI at 3.3 V [10]. The card UID is the first authentication factor; since a UID is not secret, it is combined with a PIN as the second factor. The 16-key matrix keypad uses eight lines, which are read through a PCF8574 8-bit I2C I/O expander at address 0x20 [12].

Display and actuators. The LCD1602 is driven through an I2C backpack (address 0x3F on the module used) on the same



(a) DHT11 (DHT22 model) (b) LDR (c) MQ2 module (d) PCF8591 module (e) PIR

Fig. 3. Environmental and occupancy sensors. The DHT22 model is shown for the DHT11, which has the same single-wire interface, and the HC-SR501 model is shown for the smaller HC-SR505 PIR.



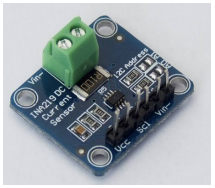
(a) RC522 reader (b) 4×4 keypad (c) PCF8574 expander

Fig. 4. Components of the two-factor access control.



(a) LCD1602 (b) Relay module (c) Servo SG90 (d) Buzzer (e) LED

Fig. 5. Display and actuators. The I2C backpack of the LCD is not shown, and the servo is a generic micro-servo model.



(a) INA219 module

Fig. 6. Measurement module (final stage).

3.3 V I2C bus as the other devices. A one-channel relay module switches only the 5 V DC fan; mains-powered appliances are excluded for safety. The SG90 micro servo is positioned by a 50 Hz PWM signal and acts as the door lock; its current peaks can reach several hundred milliamperes, so it uses the separate 5 V supply with a common ground. A buzzer sounds the gas alarm and the failed-access alarm. LEDs with 330 Ω resistors indicate the room light, the successful and failed access result, and the node mode.

Measurement. The INA219 is an I2C high-side current monitor with an on-board shunt resistor [14]. It is planned for the final stage: it is placed in series with the supply of the ESP8266 node only, so that the average current compared in RQ2 does not include the servo or the fan. Because every device of the node runs on a single 3.3 V I2C segment, no level conversion is needed on the bus.

Component images come from the Fritzing parts library (CC BY-SA 3.0) and the Wokwi Elements library (MIT license) and illustrate the module types; they are not photographs of the exact parts used. The adapter and the XY-3606 are vendor product images.

D. Hardware Interfacing Plan

Fig. 8 shows the wiring and Tables III and IV give the pin assignment. The following rules shaped it.



(a) 12 V DC adapter



(b) XY-3606 converter

Fig. 7. Power supply of the node: the 12 V adapter and the XY-3606 module that generates the 5 V rail for the relay, the fan, the MQ2 heater and the servo (common ground with the NodeMCU).

TABLE II
COMPONENTS AND INTERFACES

Component	Role	Interface	Supply	Stage
ESP8266 (NodeMCU)	MCU, Wi-Fi, local logic	–	5 V USB	OPA1
DS1307 RTC module	Time base for schedules and logs	I2C (0x68)	3.3 V	OPA1
DHT11	Temperature, humidity	1-wire	3.3 V	OPA1
LDR (CDS 5537) + 10 kΩ	Ambient light	Analog divider → PCF8591	3.3 V	OPA1
PCF8591 module	8-bit ADC for the LDR	I2C (0x48)	3.3 V	OPA1
MQ2 module	Gas and smoke	Analog, 10 k/10 k divider	5 V	OPA1
PIR (HC-SR505)	Occupancy	Digital → PCF8574 (0x21)	3.3 V	OPA1
RFID RC522	Card authentication, 13.56 MHz	SPI	3.3 V	OPA1
PCF8574 (0x21)	Outputs (buzzer, LEDs) and PIR input	I2C	3.3 V	OPA1
LCD1602 + I2C backpack	Time, readings, mode	I2C (0x3F)	3.3 V	OPA1
Relay module, 1 channel	Switches the 5 V DC fan	Digital (D0)	5 V	OPA1
Servo SG90	Door lock	PWM (D4)	5 V ext.	OPA1
Buzzer, LEDs + 330 Ω	Alarm, light, status	Expander (sink)	3.3 V	OPA1
Keypad 4×4 + PCF8574 (0x20)	PIN entry (second factor)	I2C	3.3 V	OPA2
INA219 current sensor	Node current for RQ2	I2C (0x40)	3.3 V	Final
12 V DC adapter	Input power for the 5 V rail	–	12 V out	OPA1
XY-3606 module	12 V to 5 V converter for relay, fan, MQ2, servo	–	12 V in / 5 V out	OPA1

- **One analog input.** The ESP8266 has a single ADC. The MQ2 uses it, so that the gas alarm does not depend on the I2C bus; the LDR is read through the PCF8591.
- **Pin budget.** After the I2C bus (D1, D2) and the SPI reader (D5–D8) only D0, D3 and D4 remain. They carry the relay, the DHT11 and the servo, so every usable pin of the node is allocated. The remaining digital signals (PIR input, buzzer, four LEDs) are therefore moved to a PCF8574 expander; the keypad planned for OPA2 uses a second expander on the same bus.
- **Boot pins.** GPIO0 (D3), GPIO2 (D4) and GPIO15 (D8) influence the boot mode and must not be held at the wrong level at power-up; the DHT11 module has a pull-up, and the servo line is checked for glitches at start-up. The relay is driven to its inactive state first thing in the firmware.
- **Expander outputs.** A PCF8574 pin sinks current well but sources only a weak current [12]. All loads on the expander are therefore connected as sinks (3.3 V, resistor, LED or buzzer, expander pin), and the PIR is read as an input.
- **Voltage levels.** ESP8266 GPIOs are not 5 V tolerant. The MQ2 analog output is scaled down by a 10 kΩ/10 kΩ divider and read on A0. All I2C devices (DS1307, LCD backpack, PCF8574, PCF8591) are supplied from the 3.3 V rail, so the bus has a single 3.3 V segment and no level shifter is required. An early build powered the DS1307 from 5 V; its 4.7 kΩ pull-ups then pulled SDA and SCL to 5 V and current flowed through the protection diodes of the ESP8266, so the board appeared to be powered even with USB unplugged. The RTC was moved to 3.3 V and the bus became stable.
- **Addresses.** The four I2C devices present at OPA1 (0x68, 0x3F, 0x21, 0x48) are confirmed with an I2C scanner; the PCF8574 expander is set to an address different from the LCD backpack. The keypad expander (0x20) and the INA219 (0x40) join the same bus at OPA2 and at the final stage.
- **Power.** The ESP8266 is supplied from USB; the relay coil, the fan, the MQ2 heater and the servo use a separate 5 V

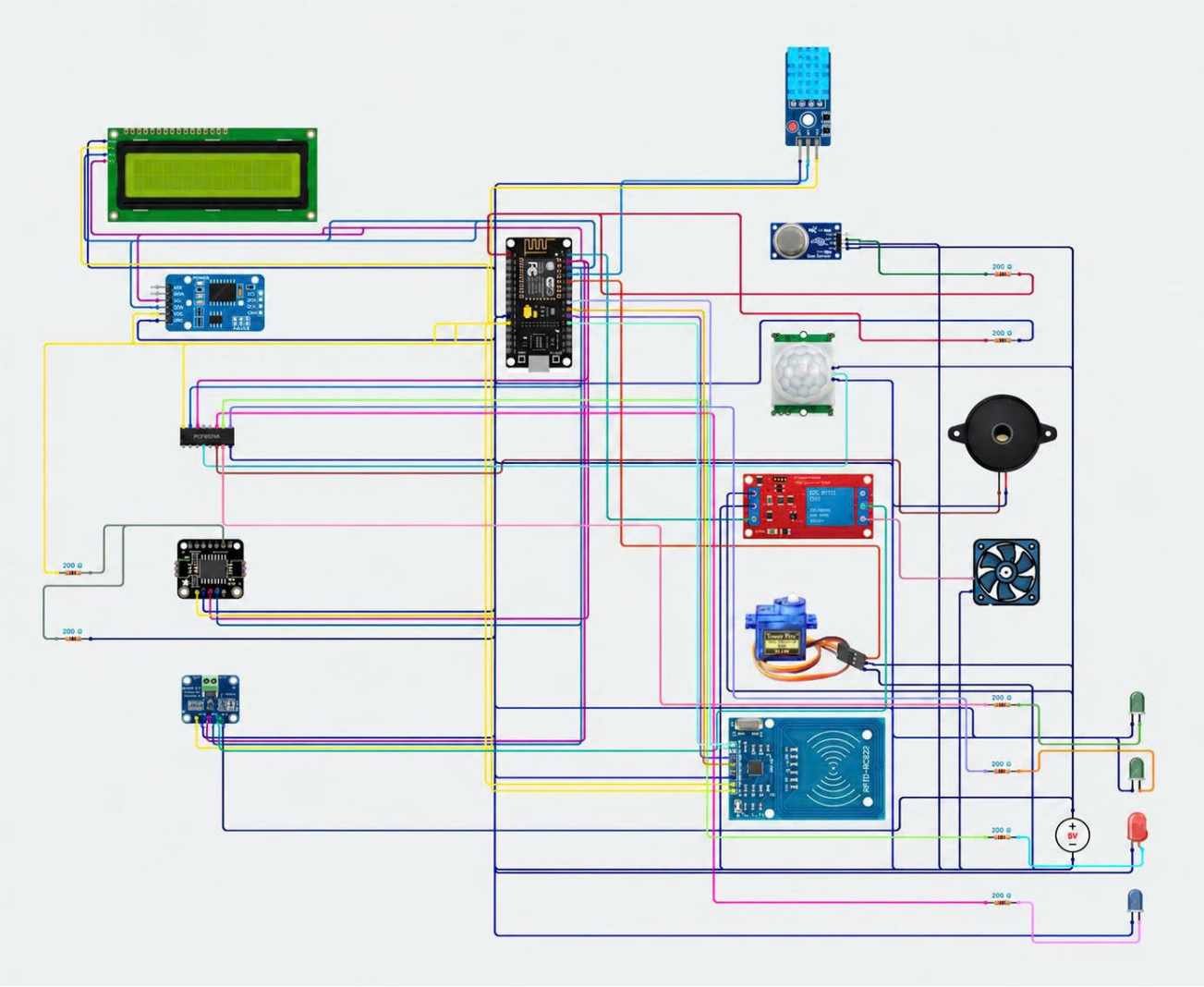


Fig. 8. Wiring diagram of the prototype.

TABLE III
ESP8266 (NODEMCU) PIN ASSIGNMENT

Signal	Pin	GPIO	Note
I2C SCL	D1	5	3.3 V I2C bus (LCD, RTC, PCF8574, PCF8591)
I2C SDA	D2	4	as above
RC522 SCK / MISO / MOSI	D5 / D6 / D7	14 / 12 / 13	SPI at 3.3 V
RC522 SDA (SS)	D8	15	boot-strapping pin
RC522 RST	3V3	–	tied high, software reset only
DHT11 DATA	D3	0	boot-strapping pin, module pull-up
Servo PWM	D4	2	boot-strapping pin; servo powered from the external 5 V
Relay IN (fan)	D0	16	digital output only; inactive state at boot
MQ2 AOUT	A0	ADC	through the 10 kΩ/10 kΩ divider

supply from an XY-3606 module (12 V input, 5 V output) fed by a 12 V adapter, with a common ground. No bulk capacitor is fitted: the node supply is independent of the servo branch, so a servo start-up transient does not disturb the controller. A 2200 μF capacitor inserted with reversed polarity was found to load the 5 V rail and was removed.

TABLE IV
I2C DEVICES AND EXPANDER PINS

Device	Address	Segment	Use	Stage
DS1307 RTC	0x68	3.3 V	time base	OPA1
LCD backpack	0x3F	3.3 V	display (0x27 on other backpacks)	OPA1
PCF8574	0x21	3.3 V	P0 PIR (input), P2 buzzer, P3 room LED, P4 OK LED, P5 FAIL LED, P6 mode LED	OPA1
PCF8591	0x48	3.3 V	AIN0 = LDR	OPA1
PCF8574	0x20	3.3 V	keypad rows and columns (second factor)	OPA2
INA219	0x40	3.3 V	current of the node supply (RQ2)	Final

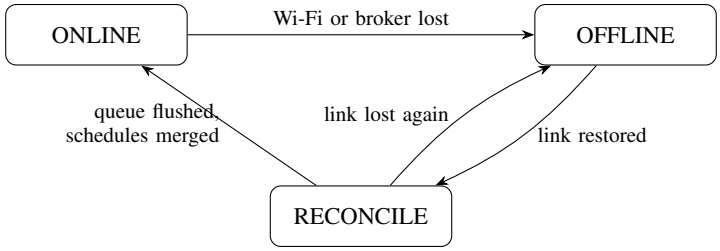


Fig. 9. Operating modes of the node.

E. Firmware Architecture

The firmware runs on the ESP8266 Arduino core. The core has no preemptive multitasking, so the design uses a cooperative loop: each function (sensing, gas monitoring, scheduling, authentication, display and networking) is called from a timer check based on `millis()`, and none of them blocks on a long `delay()`, so a slow network call cannot delay a schedule or a card read. The only bounded blocking operations are the DHT11 read and the MQTT connection attempt, whose timeout is limited to about one second and which is made only while the Wi-Fi link is up; this effect is part of what RQ1 measures. At start-up the firmware scans the I2C bus and enables only the devices that answer, so the node degrades gracefully (for example, a missing keypad expander reduces access control to the card factor). Configuration (schedules, hashed credentials, thresholds) and an event queue are kept in LittleFS flash storage, and hashing uses the SHA-256 implementation of the BearSSL library bundled with the core. At OPA1 the event queue and the hashed credentials are not yet implemented; they are added at OPA2.

F. Operating Modes

Fig. 9 shows the mode manager. In ONLINE mode the node publishes sensor data, accepts dashboard commands and keeps the RTC aligned with network time (NTP). When the Wi-Fi link or the broker is lost, the node enters OFFLINE mode: schedules, the gas alarm, the fan rules and access control continue to run from local data, and events are stored with RTC timestamps. When the link returns, the node enters RECONCILE mode, replays queued events over MQTT, exchanges schedule entries with Node-RED, which merges them with the Firebase copy, and then returns to ONLINE. The LCD and a status LED show the current mode. At OPA1 the node distinguishes only ONLINE and OFFLINE; the event queue and the RECONCILE mode are added at OPA2.

G. Schedule Reconciliation Policy

Each schedule entry carries a target (fan or light), the on/off times, a day mask, an update timestamp and its origin (device or dashboard). Reconciliation is performed by Node-RED: it compares the entries received from the node (topic `schedule/device`) with the copy stored in Firebase, keeps the entry with the newest update timestamp, writes the result back to Firebase and publishes the merged schedule to the node on the retained topic `schedule`. The node accepts an incoming entry only if its timestamp is newer than the stored one. Dashboard edits are stamped with the clock of the Node-RED host and device edits with the RTC; both clocks are NTP-synchronized when online, so a clock difference after a long outage is a possible source of conflicts and is observed in RQ1. A manual override has priority over any schedule and is logged. Every event receives an identifier and an RTC timestamp, so replays after an outage can be de-duplicated and the dashboard can show events in order. The MQTT library used on the ESP8266 publishes with QoS 0, so reliability is handled at the application level: events generated while the link is down are written to flash and replayed in RECONCILE mode, and Node-RED drops duplicates by identifier. An event published at the very moment the link fails can still be lost; this is counted as message loss in the evaluation, and an asynchronous QoS 1 client is left as future work. This policy is a design proposal that RQ1 will test, including its behavior when the same entry is edited on both sides during an outage.

TABLE V
FAN CONTROL PRIORITY

Priority	Source	Behavior
1	Gas alarm	Fan on while the alarm is active; buzzer on; event and alert published
2	Manual command	Fan on or off as commanded (Manual mode)
3	Temperature rule	On at $T \geq T_{on}$, off at $T \leq T_{off}$ (Auto mode)
4	RTC schedule (OPA2)	On inside the scheduled window of the selected days (Auto mode)

TABLE VI
MQTT TOPIC STRUCTURE (PREFIX `home/room1/`)

Topic	Direction	Content
`sensor/dht`	Node → broker	Temperature and humidity with timestamp (event-driven)
`sensor/gas`, `sensor/light`, `sensor/pir`	Node → broker	Gas level, light level, motion (value and timestamp, event-driven)
`status`	Node → broker	Online/offline (retained, last will)
`state`	Node → broker	Actuator states, modes, door, gas alarm, reporting mode, message counter, Wi-Fi signal, RTC drift (retained)
`cmd/fan`, `cmd/light`, `cmd/buzzer`	Broker → node	`on`, `off` (and `auto` for fan and light); manual override from the dashboard
`cmd/report`, `cmd/config`	Broker → node	Event or fixed reporting; temperature and gas thresholds
`access/log`	Node → broker	Access events (method, result, time)
`alert/gas`	Node → broker	Gas alarm (value, threshold, time)
`schedule`, `schedule/device` (OPA2)	Both	Schedule entries, merged by Node-RED and mirrored to Firebase
`sync/queue` (OPA2)	Node → broker	Events stored during an outage, replayed in RECONCILE mode

H. Event-Driven Reporting

The node samples sensors at a short base period and sends a value only when it differs from the last transmitted value by more than a threshold ($|\Delta T| \geq \theta_T$, $|\Delta H| \geq \theta_H$, $|\Delta G| \geq \theta_G$), or when a heartbeat interval T_{hb} has elapsed. A gas reading above the alarm threshold is sent immediately and bypasses the filter. The thresholds and T_{hb} are chosen experimentally in RQ2, and a fixed-interval mode, selected by a dashboard command, serves as the baseline.

I. Fan and Light Control

The fan is switched by four sources with a fixed priority (Table V). Gas ventilation is a safety rule and cannot be cancelled by a dashboard command. In Auto mode the fan is on when the temperature rule or the RTC schedule asks for it; the temperature rule switches on at $T \geq T_{on}$ and off at $T \leq T_{off}$, with $T_{off} < T_{on}$ to avoid chattering around the threshold, and both values can be edited from the dashboard. The room light is on when its schedule is active or when the PIR has detected motion within a hold time and the LDR reading indicates darkness; it can also be forced on or off from the dashboard.

J. Two-Factor Local Access Control

At OPA1 only the card factor is implemented: the card UID is compared with a list stored in flash. The PIN stage and the salted SHA-256 storage described below are added at OPA2. A user presents an RFID card, then enters a PIN on the keypad within a time limit. The node checks the card UID and the PIN against hashed values in local flash (a per-device salt and SHA-256), so no cloud call is needed; the PIN hash is bound to the card UID. On success the servo unlocks for a few seconds; after three failures the node sounds the buzzer and locks input for a short time. Every attempt is logged with an RTC timestamp, the method and the result. The credential list is provisioned locally through a serial command. Protection against physical tampering is outside the scope of this prototype.

K. MQTT Topics

Table VI lists the topic structure. The broker's last-will message marks the node as offline for the dashboard [11].

L. Firmware Flowchart

Fig. 10 shows one overall flowchart of the node. It is drawn with actions instead of code, so that the reader follows the behaviour of the prototype and not the implementation. The chart covers the initialisation, the network check, the sensor

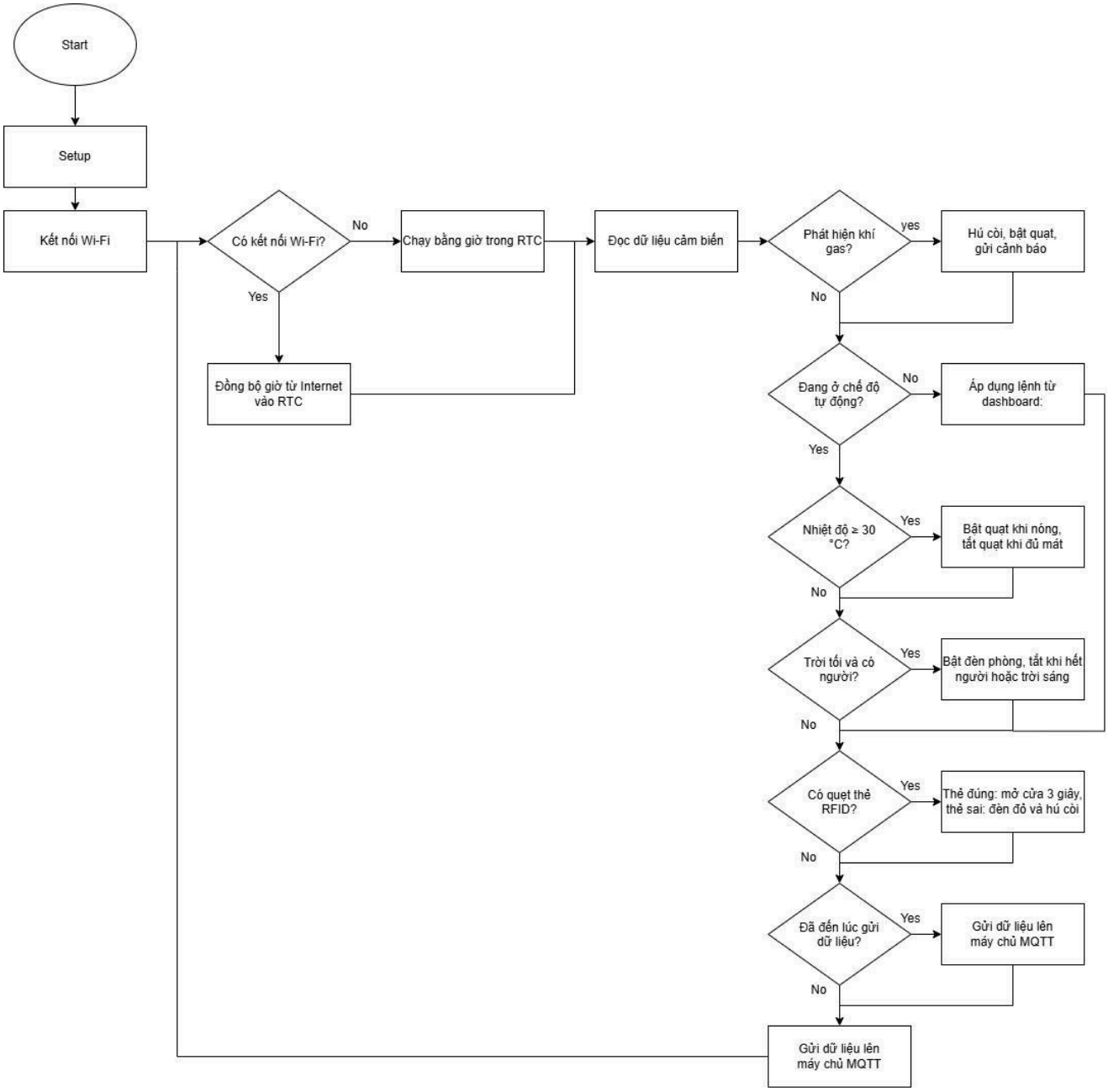


Fig. 10. Overall flowchart of the node, drawn with actions instead of code.

acquisition, the gas alarm, the manual and the automatic control path, the card access and the event-driven reporting. Two branches carry the design decisions of this section. The network check is a real branch: when the Wi-Fi link is missing the node takes the time from the RTC and keeps running, and the same holds when only the broker is lost, so the local rules never depend on the cloud. The control path follows the priority of Table V: the gas alarm overrides everything and cannot be cancelled from the dashboard, a manual command sets the fan, the light and the buzzer directly, and the temperature and light rules run only in automatic mode. The detail is kept in the text rather than in the figure: the gas rule uses a 30 s warm-up before the first alarm and a 40-count hysteresis before the alarm clears, and the fan rule keeps a 2 °C hysteresis (on at 30 °C, off at 28 °C) so that it does not chatter around one threshold. The RTC schedule, the physical mode button and the reconciliation policy are added at OPA2.

V. Prototype Status at OPA1

Table VII lists the build stages. At OPA1 the prototype is a breadboard version inside the model house; the functions marked "Built" have been wired and tested, and the others are planned for OPA2 and the final stage. Table VIII gives the functional tests performed so far, and Figs. 11–13 show the evidence. The measurements for RQ1–RQ3 have not been taken yet.

TABLE VII
BUILD STAGES OF THE PROTOTYPE

Stage	Functions	Status
OPA1 (Week 7)	ESP8266 with DHT11, LDR (via PCF8591), MQ2, PIR (via PCF8574), LCD, RTC, relay and fan, servo door lock, buzzer; a 12 V adapter with an XY-3606 5 V rail; bidirectional MQTT with the Node-RED dashboard; event-driven reporting; gas alarm with ventilation; RFID card access with a local card list; event logging to Firebase	**Built**, card factor only
OPA2 (Week 9)	Keypad and PIN (two-factor access), device-side offline queue and RECONCILE mode, RTC schedules, end-to-end validation	Planned
Final (Week 11)	INA219 energy measurement, data collection for RQ1–RQ3, finished model house, final demonstration	Planned

TABLE VIII
FUNCTIONAL TESTS AT OPA1

Test	Expected result	Result	Evidence
I2C scan	All installed devices answer at their addresses (RTC 0x68, LCD 0x3F, PCF8574 0x21, PCF8591 0x48)	Pass	Fig. 13
DHT11 reading	Temperature and humidity shown on the LCD and dashboard; within about $\pm 2\,^{\circ}\mathrm{C}$ of a reference thermometer	Pass	Fig. 12; $T = 31.1$–$33.4\,^{\circ}\mathrm{C}$, $H = 73$–$78\,\%$
MQ2 gas alarm	Buzzer on, fan on and alert published when the reading exceeds the threshold	Partial	Reading 8 in clean air and 224 near a lighter; the threshold 600 was not yet crossed
Fan control	Fan switches from the dashboard (manual) and by temperature (auto)	Pass	Relay clicks on every state change; auto above $30\,^{\circ}\mathrm{C}$
Light rule	Light on when motion is detected in the dark, off after the hold time	Partial	Illuminance index 3 (covered) to 1000 (torch); the dark decision is verified
RFID access	A registered card unlocks the servo and is logged; an unknown card is denied	Pass	`[RFID]` 2DE31C06 → `GRANTED`, servo unlocks and the event is logged; the denied path is pending
RTC	Time survives a power cycle and is aligned with NTP when online	Partial	`RTC=1`, NTP drift 0 s; the coin cell is not fitted yet
MQTT to dashboard	Sensor data and states appear on the dashboard; commands reach the node	Pass	Fig. 12; `cmd/fan` is received on the node

Rows marked *Partial* were verified only in part at the time of writing; the remaining cases are completed before the OPA2 submission.

VI. EVALUATION PLAN

Table IX defines the metrics. Experiments are repeated under normal operation and under controlled network outages (for example 1, 10 and 60 minutes; durations and repetitions to be confirmed after a pilot run). For RQ2 the same environmental scenario is applied in both reporting modes, for example a warm water cup brought near the DHT11 at fixed times, so that the readings actually change. Time references come from an NTP-synchronized computer, and all measurements are logged with timestamps. Energy is assessed from the average current of the node; battery lifetime and deep sleep are left to future work. Provisional targets from the project proposal are a mean command latency below 500 ms, message loss below 1%, at least 24 h of continuous operation, a gas alert within 2 s of crossing the threshold, and temperature readings within about $\pm 2\,^{\circ}\mathrm{C}$ of a reference thermometer.

VII. RESULTS

At OPA1 no measurements for RQ1–RQ3 were taken: the node was integrated and the functional tests of Table VIII were completed first. The measurements are collected during Weeks 8–10 and reported in Tables X–XII.

VIII. CONCLUSION AND FUTURE WORK

This report presented the design of an offline-first smart-home node that executes schedules from a hardware RTC, reports sensor data on significant change, applies local fan and light rules and verifies a card locally (a card and a PIN at OPA2),

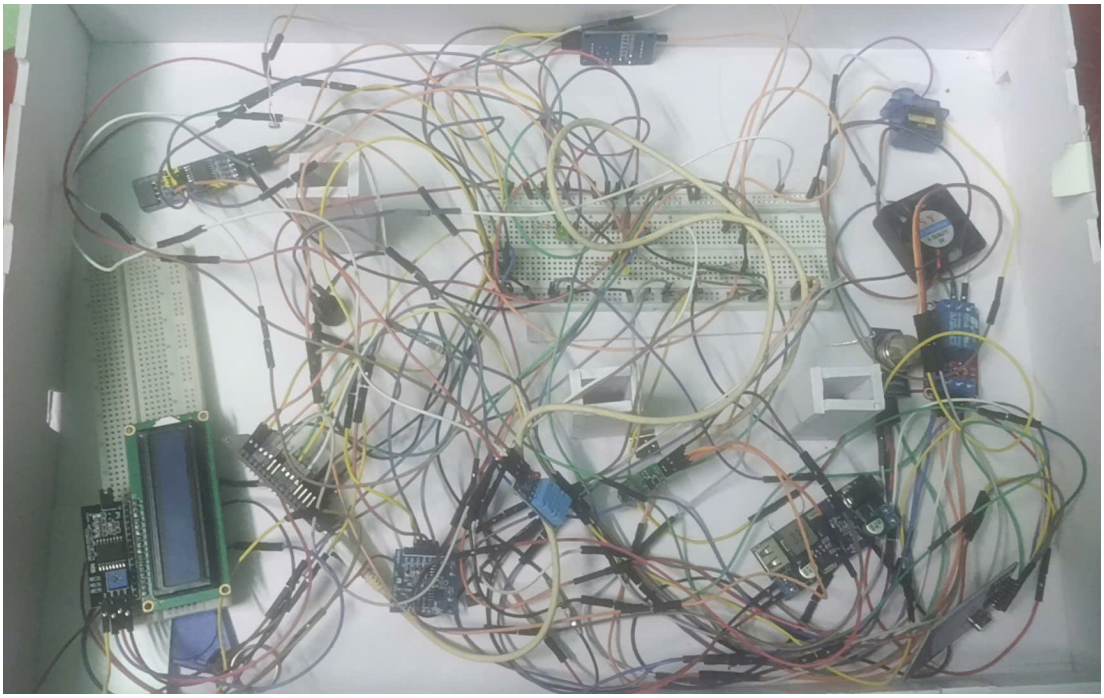


Fig. 11. Prototype inside the model house (breadboard version).

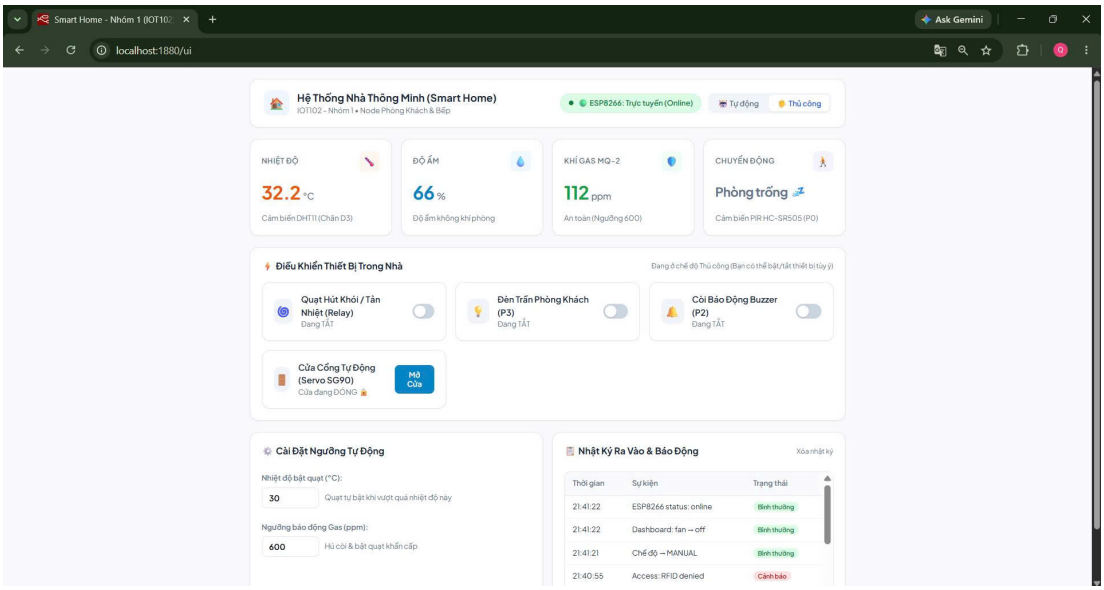


Fig. 12. Web dashboard showing live sensor data and device states.

while synchronizing with the cloud when a connection exists. The design, interfacing plan and evaluation plan define how the three research questions will be tested on a low-cost ESP8266 prototype. At OPA1 the node is integrated and all functional tests of Table VIII pass or pass in part. The next steps are the PIN factor, the device-side offline queue and the RECONCILE mode, RTC schedules, data collection and the analysis of results. Possible future work includes an asynchronous QoS 1 MQTT client, a temperature-compensated RTC, protection against physical tampering, battery operation with deep sleep, and several nodes sharing one broker.

REFERENCES

[1] A. L. Haroon P. S. *et al.*, "Home automation using Wi-Fi: ESP32-based system for remote control and environmental monitoring," in *Proc. 3rd Int. Conf. Distributed Computing and Electrical Circuits and Electronics (ICDCECE)*, Ballari, India, Apr. 2024, pp. 189–193.

[2] F. D. Makatita and N. F. A. Hakim, "MQTT protocol-based ESP-32 smarthome with multi-sensor recognition," *J. Electr., Electron., Inf., Commun. Technol. (JEEICT)*, vol. 6, no. 1, pp. 29–36, Apr. 2024.

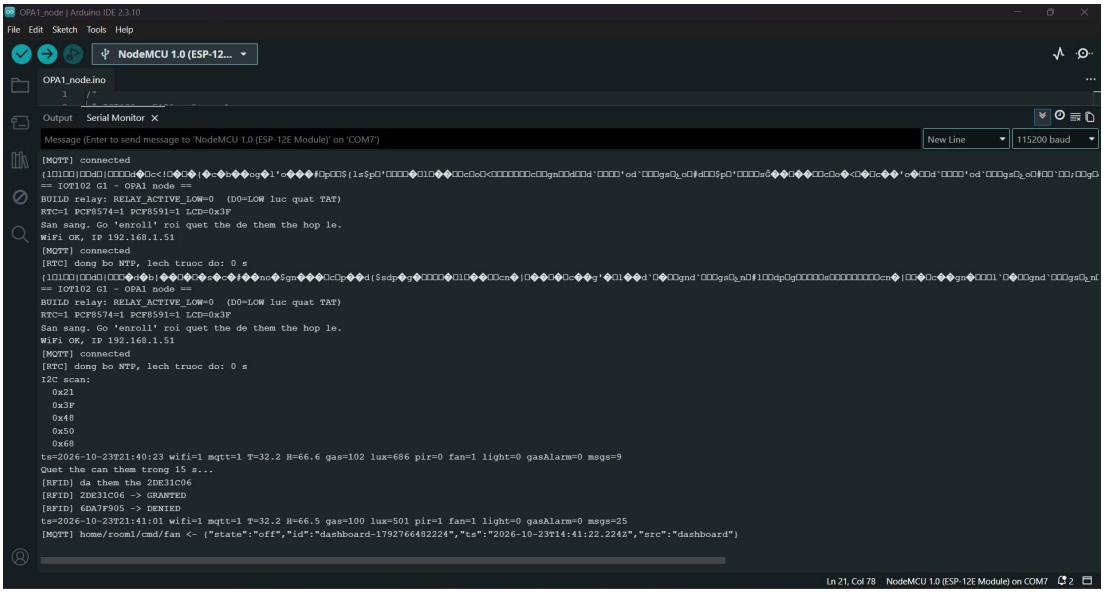


Fig. 13. Serial monitor output of the I2C scan and the node status.

TABLE IX
EVALUATION METRICS

RQ	Metric	Method	Instrument
RQ1	Deviation between scheduled and actual actuation time (s), online vs. offline	Compare logged actuation time with the schedule and a reference clock	Serial/MQTT logs, NTP reference
RQ1	Share of schedules executed on time during outages; time to resynchronize; number of conflicts; RTC drift at each synchronization	Repeated outages with edits on both sides	Logs, Node-RED, Firebase copy
RQ2	Messages sent, fixed vs. event-driven (%)	Same environmental scenario in both modes	MQTT broker log
RQ2	Average current (mA) and estimated energy per hour; event detection delay	Run both modes with the same sensors and scenario	INA219 current sensor
RQ3	Authentication response time (s) and success rate during outage	Repeated attempts with valid and invalid cards and PINs	Serial timestamps, access log
All	Command latency, message loss, uptime, gas alert time, time from gas threshold to fan on, sensor error	Targets in the proposal	Logs, reference thermometer

TABLE X
RQ1: SCHEDULING DEVIATION (TO BE COMPLETED)

Condition	Trials	Mean dev. (s)	Max dev. (s)	On-time (%)
Online	–	–	–	–
Offline 1 min	–	–	–	–
Offline 10 min	–	–	–	–
Offline 60 min	–	–	–	–

TABLE XI
RQ2: FIXED VS. EVENT-DRIVEN REPORTING (TO BE COMPLETED)

Mode	Messages / h	Avg. current (mA)	Detection delay (s)
Fixed interval	–	–	–
Event-driven	–	–	–

[3] M. A. Sulimat and N. Katiran, “IoT based air quality monitoring system using LoRaWAN,” *Evol. Electr. Electron. Eng.*, vol. 5, no. 2, pp. 116–125, 2024.

TABLE XII
RQ3: OFFLINE AUTHENTICATION (TO BE COMPLETED)

Condition	Attempts	Success (%)	Mean response (s)
Online	–	–	–
Offline	–	–	–

[4] A. Parkavi *et al.*, "Air quality and dust level monitoring systems in hospitals using IoT," *Discover Internet of Things*, vol. 5, Art. no. 23, 2025.
[5] A. A. Alzhrani *et al.*, "Design and implementation of an IoT-integrated smart locker system utilizing facial recognition technology," *Eng. Technol. Appl. Sci. Res.*, vol. 14, no. 4, pp. 16000–16010, Aug. 2024.
[6] J. Parab and S. Kamat, "IoT based smart biometric locker," *SSRN*, Dec. 2022.
[7] M. N. Alfazri, A. Taqwa, and S. Suroso, "Design and construction of an Internet of Things (IoT)-based package receiving box security system," *Teknika: Jurnal Sains dan Teknologi*, vol. 21, no. 2, 2025, doi: 10.62870/tjst.v21i2.34883.
[8] Espressif Systems, "ESP8266EX datasheet," datasheet. [Online]. Available: https://www.espressif.com
[9] Maxim Integrated, "DS1307 64×8 serial I2C real-time clock," datasheet.
[10] NXP Semiconductors, "MFRC522 standard performance MIFARE and NTAG frontend," datasheet.
[11] OASIS, "MQTT version 3.1.1," OASIS Standard, Oct. 2014.
[12] NXP Semiconductors, "PCF8574 remote 8-bit I/O expander for I2C-bus," datasheet.
[13] NXP Semiconductors, "PCF8591 8-bit A/D and D/A converter," datasheet.
[14] Texas Instruments, "INA219 zero-drift, bidirectional current/power monitor with I2C interface," datasheet.

AI USAGE DECLARATION

Group: 1 (IA2103) Topic: Offline-First Smart Home Node **Members:** Le Xuan Quyet (SE203751), Nguyen Quoc Viet (SE211888), Nguyen Hoang Quan (SE204702), Tran Quoc Huy (SE196671)

☐ We did not use generative AI tools in this report.

☒ We did use AI. Tool(s): **Gemini**. Purposes: ☒ system/architecture design exploration; ☒ hardware/protocol selection research; ☒ code generation/debugging; ☐ data analysis support; ☐ literature exploration; ☒ technical writing/language editing; ☐ other: .

We confirm that all engineering decisions, evaluation results and conclusions in this report are the product of our own work.

TABLE XIII
AI AUDIT LOG ENTRIES

Phase & AI Suggestion	Evaluation & Testing	Detailed Technical Decision
Entry 1: System Design & Architecture (Week 4) *AI suggestion:* Connect all sensors and peripherals directly to the GPIO pins of the ESP8266.	**Pinout analysis:** The ESP8266 has only one Analog pin (A0) and a very limited number of usable Digital pins (GPIO0, GPIO2, GPIO15 are boot-strapping pins). Plugging the LDR, MQ2, 4x4 Keypad, LCD1602, and buzzer/LEDs directly causes a severe pin shortage.	**Hardware solution:** Use PCF8574 (I2C) to expand pins for the Keypad and digital outputs (Buzzer, LEDs). Use PCF8591 (8-bit ADC) to read the LDR via I2C. Keep A0 directly connected to the MQ2 via a 10k/10k voltage divider so the gas alarm operates independently of the I2C bus.
Entry 2: Programming & Firmware (Week 6) *AI suggestion:* Generate sample code to read the DHT11 and scan the keypad using sequential loops containing blocking functions like delay(1000).	**Firmware testing:** Since the ESP8266 lacks preemptive multitasking, delay() blocks the microcontroller, leading to missed RFID card reads (RC522) and delayed interrupt responses during gas incidents.	**Software architecture:** Switch to a Cooperative Loop model based on the non-blocking millis() timer. All functions (sensor reading, RTC scheduling, RFID/PIN reading, LCD updating) are checked periodically without halting the system.
Entry 3: OPA1 Assessment (Week 7) *AI suggestion:* Suggest the ESP8266 make direct HTTPS REST API/TLS Client connections to the Firebase Realtime Database to save logs and sync schedules.	**Hardware RAM evaluation:** The ESP8266 has only about 80 KB of RAM. Running the TLS/HTTPS encryption stack consumes significant memory, risking a heap overflow and system crash during network drops.	**Decoupling:** The ESP8266 only runs a lightweight MQTT client connecting to the Mosquitto broker. Node-RED acts as the sole bridge responsible for Firebase synchronization via HTTPS, keeping the ESP8266 firmware simple and ensuring local autonomy.
Entry 4: OPA2 Assessment (Week 9) *AI suggestion:* Propose that upon every RFID card swipe or PIN entry, the ESP8266 sends a request to the Cloud/Firebase Database to verify credentials.	**Wi-Fi outage scenario testing:** Without access to the Cloud database, the system immediately rejects authentication and locks users out.	**Offline-First design:** Store a list of hashed passwords (SHA-256 with per-device salt) directly in the local Flash memory (LittleFS). All 2-factor authentication operations are verified locally, meeting the Offline-first operational standard.
Entry 5: Final Evaluation (Week 11) *AI suggestion:* Configure the node to send all environmental data (Temperature, Humidity, Light, Gas) to the Dashboard at fixed intervals (30s or 120s).	**Energy evaluation:** Fixed-interval reporting wastes Wi-Fi radio energy unnecessarily when environmental parameters are static.	**Energy optimization at the Edge:** Switch to an Event-driven (Threshold-based) reporting mechanism. Data is only published when the change exceeds a threshold ($\|\Delta T\| \geq \theta_T$, $\|\Delta H\| \geq \theta_H$, $\|\Delta G\| \geq \theta_G$) or the Heartbeat interval expires.